



开放数据中心标准推进委员会
Open Data Center Committee

[编号 ODCC-2021-03003]

S³IP –PIT 规范

(Simplified Switch System Integration Program-Platform Integration Test Specification)

www.ODCC.org.cn

开放数据中心标准推进委员会

2021-06-30 发布

目录

前 言	II
引 言	III
1 范围	1
2 规范性引用文件	2
3 术语、定义和缩略语	3
4 PIT 规范	5
4.1 基本要求	5
4.2 EERPOM 内容测试	5
4.3 CPU MAC 地址检查	9
4.4 BMC MAC 地址检查	9
4.5 系统固件版本	10
4.6 固件更新和切换	12
4.7 系统 Sensors	17
4.8 CPU 系统测试	18
4.9 可替换部件	25
4.10 管理网口	28
4.11 光模块和 DAC	29
4.12 数据平面	30
4.13 逻辑器件测试	30
本规范用词说明	32

前 言

本规范共分 4 章，主要技术内容有：范围、规范性引用文件、术语、PIT (Platform Integration Test) 规范。

ODCC-2021-03003 文档主要描述 PIT 规范，提供产测和验收手段，检验设备是否符合 S³IP 规范，隶属于 S³IP 软硬件规范系列之一。S³IP 软硬件规范还包括 Sysfs 规范 (开源 SONIC 和 FW 的 API 接口规范化)，以及硬件基础功能规范，提供底层能力支撑 API 接口标准化落地。

本规范由 S³IP 项目组负责管理和具体技术内容的解释，PIT 规范主要作者为李华。本规范在执行过程中，请各单位结合工程实践，认真总结经验，如发现需要修改或补充之处，请将意见和建议 Mail 至 fangbo.zfb@alibaba-inc.com。

以供今后修订时参考。

主编单位： 百度、阿里、腾讯、美团、京东、快手

参编单位、主要审核人（排名按照姓名字母排序）： 崔鹏(腾讯)、陈虎(美团)、程传胜(腾讯)、杜海峰(快手)、黄一元(阿里)、李华(阿里)、李竞(百度)、沈大勇(腾讯)、申路(美团)、孙勇峰(腾讯)、田上飞(腾讯)、王超(阿里)、文权(腾讯)、王家富(京东)、吴教仁(快手)、王永灿(阿里)、夏云磊(京东)、杨光(腾讯)、朱芳波(阿里)、郑磊(美团)、占志敏(阿里)

引 言

随着互联网的高速发展，数据中心的规模也随之爆发式增长，全球数据中心总数量几十万，超大型数据中心数量几百个。数据中心内部、不同数据中心之间海量数据互联，Machine to Machine以及Machine to User的高带宽、低延迟的持续提升需求，需要依靠网络实现，作为承担网络发展的载体交换机，近几年市场出现一些明显变化：1、互联网客户纷纷自研白盒交换机、构建自己的基础设施能力，针对各自应用场景“精”“简”化设计，研发模式基本采用轻量化模式；2、中国互联网客户非常默契选用开源SONIC作为交换机OS，构建各家自研网络。轻量化研发模式需要将交换机重投入的底层技术如硬件、驱动、FW交给ODM/OEM实现，ODM/OEM提供白盒设备给互联网公司porting各自OS上线。不同厂商设备之间设计存在差异，导致OS porting不同厂商设备花费资源较多；ODM/OEM厂商提供得底层驱动/FW融合了多个客户需求，导致非互联网客户必须的需求特性被带入互联网的应用场景，对线上稳定性会产生影响，对于互联网客户而言始终是个隐患。通过标准化交换机OS和底层驱动/FW/硬件基础设计的接口部分设计，可以简化OS porting厂商设备难度，降低互联网客户资源投入；同时，国内互联客户OS全切换到SONIC后，标准化OS和底层FW/驱动/硬件基础功能之间接口部分设计，屏蔽不同厂商设备之间的差异，技术完全可行。

本规范提出PIT规范，用于规范化交换机软硬件设计规范。

本规范旨在：通过本标准引导互联网白盒交换机设计，统一软硬件设计规范，实现上层OS porting白盒化交换机便利化；同时，S³IP项目组成员部署基于同一标准设计的交换机，扩大标准交换机应用规模，能更容易发现和解决问题，提升设备上线稳定性。

S³IP-PIT 规范

1 范围

本规范提出交换机的软硬件技术规范，内容如下：

- PIT规范；

自2020年11月开始S³IP在ODCC立项，BAT+美团+京东+快手进行了将近一年的规范讨论和研究，本标准涉及内容当前均已在各家自研设备上落地，得到了验证，但并未统一成六家共同遵守的规范。通过S³IP项目，后续各家自研交换机将围绕统一的标准规范进行设计。

本规范是集体智慧的结晶，希望规范中不当之处得到业界同仁的指正，编者将吸取大家的宝贵意见再次修订本规范，期待本规范再版时更加完整、严谨、丰富。

www.ODCC.org.cn

2 规范性引用文件

下列文件对于本文件的应用是必不可少的，凡是注日期的引用文件，仅注日期的版本适用于本文件。凡是不注日期的引用文件，其最新版本（包括所有的修改单）适用于本文件。

1. RFC2544 Benchmarking Methodology for Network Interconnect Devices.
2. IA OIF-CMIS Common Management Interface Specification.
3. SFF-8024 SFF Module Management Reference Code Tables.
4. SFF-8636 Management Interface for 4-lane Modules and Cables
5. SFF-8679 QSFP+ 4X Hardware and Electrical Specification
6. Common Management Interface Specification (CMIS)
7. FRU Spec. Platform Management FRU Information Storage Definition v1.0

www.ODCC.org.cn

3 术语、定义和缩略语

数据中心 Data center

为集中放置的电子信息技术设备提供运行环境的建筑场所，可以是一栋或几栋建筑物也可以是一栋建筑物的一部分，包括主机房、辅助区、支持区和行政管理区等。

交换机 Switch

一种用于电（光）信号转发的网络设备。它可以为接入交换机的任意两个网络节点提供独享的电信号通路。

操作系统 OS

操作系统 (operation system, 简称 OS) 是管理计算机硬件与软件资源的计算机程序。操作系统需要处理如管理与配置内存、决定系统资源供需的优先次序、控制输入设备与输出设备、操作网络与管理文件系统等基本事务。操作系统也提供一个让用户与系统交互的操作界面。

光端口 Optial Fiber Connector

光纤接口简称，应用于机房、机柜等大型设备的一个光纤带宽接口。

配置接口 Console

插入 Console 线可直接连接至交换机，对交换机进行配置。

Host CPU

交换机系统中的主控 CPU，用于管理和控制主交换芯片。

BMC

Baseboard Management Controller，在交换机系统中负责系统管理和控制，监控温度、电源等系统状态并负责异常告警和处理。独立于 Host CPU。

二次电源

这里指交换机系统中，各类芯片所需的直流低压电源，区别于系统外部供电的 220V

交流或者高压直流电源。通常使用 DC/DC 芯片把 PSU 输出的 12V 电压转变为芯片所需的各种低电压。

PSU

Power Supply Units (PSU), 交换机电源模块, 用于将 220V 交流或高压直流输入转变成 12V 直流电压。

CPLD

是 Complex PLD 的简称, 顾名思义, 其是一种较 PLD 为复杂的可编程逻辑器件。采用 CMOS EPROM、EEPROM、FLASH 和 SRAM 等编程技术, 构成了高密度、高速度和低功耗的可编程逻辑器件。

FPGA

Field Programmable Gate Array 的缩写, 即现场可编程门阵列, 它是在 PAL、GAL、EPLD 等可编程器件的基础上进一步发展的产物。用户可对 FPGA 内部的逻辑模块和 I/O 模块重新配置, 以实现用户的逻辑。FPGA 的编程数据放在 Flash 芯片中, 通过上电加载到 FPGA 中, 对其进行初始化。

TLV

TlvInfo EEPROM, ONIE 兼容的交换机系统所需要支持的 EEPROM 信息格式。存储的数据包括分配给系统的 MAC 地址、序列号、制造日期等信息。

平台, 功能等同于 Diag。

www.ODCC.org.cn

4 PIT 规范

4.1 基本要求

4.1.1 PIT (Platform Integration Test) 是自研交换机系统诊断工具, 应该支持三种模式: 研发测试, 工厂生产测试, 现场交付测试。

4.1.2 PIT 测试的读取和设置, 如果存在 sonic 命令行, 优先使用, 否则, 使用每个平台厂商自定义的命令行; 如果测试项是必选项, 缺失的命令, 应该标准化该命令的实现。

4.1.3 PIT 的测试新增的命令, 存放在/usr/local/bin 目录。

4.2 EERPOM 内容测试

4.2.1 CPU 端 TLV-EEPROM 内容读取测试:

a)、测试方法: 通过 SONIC 命令行读取, show platform syseeprom.

b)、判断标准: 命令执行成功和返回值满足如下特定标准

(1): Key 检查, 必需包含["Product Name", "Part Number", "Serial Number", "Base MAC Address", "MAC Addresses", "Platform Name", "Vendor Name", "Manufacturer"].

(2) Value 检查, MAC 地址合法性检查, PartNumber, SerialNumber, PlatformName 等, 非空字符串且不能是全部字符相.

4.2.2 CPU 端 TLV-EEPROM 内容烧写测试:

a)、测试工具: syseeprom-set 工具, 支持 TLV-EEPROM 烧写.

b)、测试方法：依次执行下述步骤(1)~(5)

(1) 通过 SONIC 命令行读取, show platform syseeprom, 记录为原始 TLV

eeeprom, syseeprom_orig.

(2) 通过 syseeprom-set 写入测试版 TLV eeeprom 内容 syseeprom_test, 并

且再次读取, 记录为测试版读回 TLV eeeprom, syseeprom_test_readback.

(3) 对比 syseeprom_test 和 syseeprom_test_readback.

(4) 通过工具把原始版 syseeprom_orig 写入 TLV eeeprom, 再次读取, 记录

为 syseeprom_orig_readback.

(5) 对比 syseeprom_orig 和 syseeprom_orig_readback.

b)、判断标准:

(1) 所有读取, 烧写命令执行成功.

(2) 上述对比步骤(3), (5)内容对比返回无差异.

4.2.3 板卡级 EEPROM 读取测试:

a)、适用范围: 针对不同产品的子板卡适用, 包括主板, CPU 板, 交换板, BMC 板,

线卡, 风扇板, 系统 SYS_EEPROM, 等等. EEPROM 内容遵循 FRU-EEPROM 格式.

b)、测试命令工具: fruidutil get 命令, 支持 baseboard, cpu, switch, bmc, linecard,

fanctrl, sys, fan, psu 这些参数, 根据硬件差异, 可选

c)、测试方法: 使用 fruidutil get 读取 FRU EEPROM

d)、判断标准: 命令执行成功和返回值满足如下特定标准

- (1) Key 检查, 必需包含["Product Name", "Part Number", "Serial Number", "Manufacturer"].
- (2) Value 检查, PartNumber, SerialNumber, ProductName, Manufacturer 等, 非空字符串且不能是全部字符相同.

4.2.4 板卡级 EEPROM 烧写测试:

- a)、适用范围: 针对不同产品的子板卡适用, 包括主板, CPU 板, 交换板, BMC 板, 线卡, 风扇板, 系统 SYS_EEPROM, 等等. EEPROM 内容遵循 FRU-EEPROM 格式.
- b)、测试命令工具: fruutil get 命令, 支持 baseboard, cpu, switch, bmc, linecard, fanctrl, sys, fan, psu 这些参数, 根据硬件差异, 可选.
- c)、测试方法: 依次执行下述步骤(1)~(5)

(1) 通过 fruutil get 读取, 记录\$(board_type)_eeprom_orig.

(2) 通过 fruutil set 写入测试版 eeprom 内容\$(board_type)_eeprom_test,

并且再次读取, 记录为测试版读回 eeprom, \$(board_type)_eeprom_test_readback.

(3) 对比\$(board_type)_eeprom_test 和\$(board_type)_eeprom_test_readback

(4) 过工具把原始版\$(board_type)_eeprom_orig 写入 eeprom, 再次读取, 记录为\$(board_type)_eeprom_orig_readback.

(5) 对比\$(board_type)_eeprom_orig 和\$(board_type)_eeprom_orig_readback.

d)、判断标准:

(1) 所有读取, 烧写命令执行成功.

(2) 上述对比步骤 (3), (5)内容对比返回无差异.

4.2.5 可替换部件 EEPROM 读取测试:

a)、适用范围: 风扇, PSU EEPROM 内容遵循 FRU-EEPROM 格式.

b)、测试命令工具: fruutil get 命令, 支持 baseboard, cpu, switch, bmc, linecard, fanctrl, sys, fan, psu 这些参数, 根据硬件差异, 可选.

c)、测试方法: 依次执行下述步骤(1)~(5)

(1) 通过系统工具行读取, 记录为原始 eeprom, \$(fru_type)_eeprom_orig.

(2) 通过产品提供的工具写入测试版 eeprom 内容\$(fru_type)_eeprom_test, 且再次读取, 记录为测试版读回 eeprom, \$(fru_type)_eeprom_test_readback.

(3) 对比\$(fru_type)_eeprom_test 和\$(fru_type)_eeprom_test_readback.

(4) 通过工具把原始版\$(fru_type)_eeprom_orig 写入 eeprom, 再次读取, 记录为\$(fru_type)_eeprom_orig_readback.

(5) 对比\$(fru_type)_eeprom_orig 和\$(fru_type)_eeprom_orig_readback.

d)、判断标准: 命令执行成功和返回值满足如下特定标准

(1) Key 检查, 必需包含["Product Name", "Part Number", "Serial Number", "Manufacturer"].

(2) Value 检查, PartNumber, SerialNumber, ProductName, Manufacturer 等,

非空字符串且不能是全部字符相同.

4.3 CPU MAC 地址检查

4.3.1 MAC 地址分配规则

TLV-EEPROM 提供了 BaseMac 和 MacCount 两个属性, CPU MAC 是 BaseMAC, 分配给 CPU 的管理网口 eth0.

4.3.2 测试方法:依次执行步骤 1) ~3)

- 1) 读取 TLV eeprom 内容, 获取 Base MAC
- 2) 根据 MAC 地址分配规则, 生成 CPU MAC 期望值
- 3) 用 Linux 命令 ifconfig 查看 CPU 管理网口 MAC 地址

4.3.3 判断标准

- 1) 命令执行都成功
- 2) 生成的 CPU MAC 和读取到的 CPU 管理网口 MAC 地址相同

4.4 BMC MAC 地址检查

4.4.1 MAC 地址分配规则

TLV-EEPROM 提供了 BaseMac 和 MacCount 两个属性, BMC MAC 是 BaseMAC+1, 分配给 BMC 的管理网口 eth0.

4.4.2 测试方法

- 1) 读取 Syseeprom 内容, 获取 Base MAC
- 2) 根据 MAC 地址分配规则, 生成 BMC MAC 期望值

3) 用 Linux 命令 `ifconfig` 查看 BMC 管理网口 MAC 地址

4.4.3 判断标准

1) 命令执行都成功

2) 生成的 BMC MAC 和读取到的 BMC 管理网口 MAC 地址相同

4.5 系统固件版本

4.5.1 固件版本管理：

a)、固件版本管理原则：出厂固件版本基线固定，后续生产导入的版本必须符合基线版本或者更新版本。

b)、适用范围：所有 CPLD, FPGA, PCIe 固件版本以及 BMC 版本。

c)、测试方法：依次执行如下步骤：读取固件版本，记录为\$(fw_type)_running；对比\$(fw_type)_running 和\$(fw_type)_baseline

d)、判断标准：

(1) 所有读取命令成功。

(2) \$(fw_type)_running 和\$(fw_type)_baseline 测试执行结果应该一致。

4.5.2 PIT 版本和 Sonic 版本：

a)、管理原则：PIT 程序对 Sonic 版本可能存在依赖，要求 Sonic 版本高于某个基线版本。出厂 PIT 版本和 Sonic 版本都需要有一个基线版本号。

b)、测试方法：依次执行下属步骤(1)~(5)

(1) 用 `show version` 命令读 Sonic 版本号，记为 `sonic_version_running`。

(2) 对比 sonic_version_running 和 Sonic 基线版本号 sonic_version_baseline.

(3) 用 sysdiag --version 命令读取 diag 程序版本, 记为 diag_version_running.

(4) 对比 diag_version_running 和 diag_version_baseline.

(5) 对比 diag_version_running 和 diag_sonic_version.

c)、判断标准:

(1) 所有读版本命令执行成功.

(2) 上述步骤(2), (4)需要满足: running 版本比 baseline 版本相同.

(3) 上述步骤(5)需要满足: running 版本比 baseline 更新或者相同.

4.5.3 BIOS 版本:

a)、适用范围: X86 平台.

b)、测试方法: 用 dmidecode -t bios 命令读取

c)、判断标准:

(1) 命令执行成功.

(2) 检查版本信息必须包含 Version 字段, value 记为 bios_version_running.

(3) 对比 bios_version_running 和 bios_version_baseline, 必须相同或者比 baseline 更新.

4.5.4 UBOOT 版本:

a)、适用范围: ARM, PPC 平台.

b)、测试方法: 用 fw_printenv 命令读取

c)、判断标准:

(1) 命令执行成功.

(2) 检查版本信息必须包含 Version 字段, value 记为 uboot_version_running.

(3) 对比 uboot_version_running 和 uboot_version_baseline, 必须相同或 baseline 更新.

4.5.5 交换芯片 SDK 版本:

a)、测试方法: 用芯片 SDK 提供的工具命令读取 SDK 版本号

b)、判断标准:

(1) 命令执行成功.

(2) 芯片 SDK 版本 sdk_version_running 和 sdk_version_baseline, 相同或者更新.

4.6 固件更新和切换

4.6.1 CPLD 更新:

a)、测试方法:依次执行步骤(1)~(5)

(1) 遍历所有 CPLD, 依次执行: 读当前版本, 记为 \$(cpld_name)_version_running;以及用平台/产品工具升级 CPLD (该 CPLD 固件是 test 版本) .

(2) 执行 CPLD 固件 refresh.

(3) 遍历所有 CPLD, 依次执行: 读当前版本, 记为
\$(cpld_name)_version_test_readback ; 对 比
\$(cpld_name)_version_test_readback 和 \$(cpld_name)_version_test;
用平台/产品工具升级 CPLD 到原始版本.

(4) 执行 CPLD 固件 refresh.

(5) 遍历所有 CPLD, 依次执行: 读当前版本, 记为
\$(cpld_name)_version_running_readback ; 对 比
\$(cpld_name)_version_running_readback 和
\$(cpld_name)_version_running.

b)、判断标准:

(1) 对上述所有操作, 执行成功.

(2) 对比\$(cpld_name)_version_test_readback 和 \$(cpld_name)_version_test
版本一致.

(3) 对比\$(cpld_name)_version_running_readback 和
\$(cpld_name)_version_running 版本一致

4.6.2 FPGA 更新:

a)、测试方法:依次执行步骤(1)~(9)

(1) 读取当前 FPGA 版本, 记为 fpga_version_running.

(2) 升级 fpga 固件到 test 版本.

(3) 执行 FPGA 固件 refresh.

(4) 读取当前 FPGA 固件版本, 记为 fpga_version_test_readback.

(5) 对比 fpga_version_test 和 fpga_version_test_readback.

(6) 升级 fpga 固件到原始版本.

(7) 执行 FPGA 固件 refresh.

(8) 读取当前运行 FPGA 固件版本, 记为 fpga_version_running_readback.

(9) 对比 fpga_version_running 和 fpga_version_running_readback.

b)、判断标准:

(1) 对上述所有操作, 命令执行成功.

(2) 步骤(5), (9)对比, 版本一致.

4.6.3 交换芯片 PCIe 固件更新:

a)、测试方法:依次执行步骤(1)~(7)

(1)用交换芯片 SDK 工具读取当前 PCIe 固件版本, 记为 pcie_version_running.

(2) 升级 PCIe 固件到 test 版本.

(3) 读取当前 PCIe 固件版本, 记为 pcie_version_test_readback.

(4) 对比 pcie_version_test 和 pcie_version_test_readback.

(5) 升级 PCIe 固件到原始版本.

(6) 读取当前运行 PCIe 固件版本, 记为 pcie_version_running_readback.

(7) 对比 pcie_version_running 和 pcie_version_running_readback.

b)、判断标准:

(1) 对上述所有操作, 命令执行成功.

(2) 步骤(4), (7)对比, 版本一致.

4.6.4 BIOS 更新和切换:

a)、测试方法:依次执行步骤(1)~(14)

(1) 用 dmidecode -t bios 读取当前 bios 版本, 记为 bios_version_running.

(2)用平台/产品提供的工具升级 bios, 指定主 flash, 版本为 bios_version_test.

(3) 重启.

(4) 启动完成后读取 bios 版本, 记为 bios_version_test_readback.

(5) 对比 bios_version_test 和 bios_version_test_readback.

(6) 升级 bios, 指定备 flash, 版本为 bios_version_test_slave.

(7) 通过平台/产品工具切换 bios 到备用 flash.

(8) 重启.

(9) 启动完成后读取 bios 版本, 记为 bios_version_test_slave_readback.

(10) 对比 bios_version_test_slave 和 bios_version_test_slave_readback.

(11) 升级 bios, 指定主 flash, 版本为 bios_version_running.

(12) 重启.

(13) 启动完成后读取 bios 版本, 记为 bios_version_running_readback.

(14) 对比 bios_version_running 和 bios_version_running_readback.

b)、判断标准:

(1) 对上述所有操作, 命令执行成功.

(2) 步骤(5), (10),(14)对比, 版本一致.

4.6.5 BMC 更新和切换:

a)、测试方法:依次执行步骤(1)~(8)

(1) 通过平台/产品工具读取 BMC 当前版本, 当前 flash, 记录为当前 bmc_version_running, bmc_flash_running.

(2) 通过平台/产品工具升级 BMC standby flash.

(3) 通过平台/产品工具切换到 BMC standby flash.

(4) 通过平台/产品工具重启 BMC.

(5) 重启完成后读取 BMC 版本和 flash, 记为 bmc_version_test_readback, bmc_flash_test_readback.

(6) 对比 bmc_version_test 和 bmc_version_test_readback.

(7) 对比 bmc_flash_test_readback 和 bmc_flash_running.

(8) 重复操作步骤 (1)~步骤 (6).

b)、判断标准:

(1) 对上述所有操作, 命令执行成功.

(2) 步骤(5)对比, 版本一致.

(3) 步骤 6 对比, flash 不一致

4.7 系统 Sensors

4.7.1 温度 sensor:

a)、测试方法:依次执行步骤(1)~(3)

(1) 读取系统温度, 包括 CPU 温度, 交换芯片温度, 进风口/出风口温度,内存温度, 记为\$(temp_type)_value.

(2) 读取系统温度相应的阈值, 记为 \$(temp_type)_warning, \$(temp_type)_alarm.

(3) 遍历系统温度, 对比当前值\$(temp_type)_value和\$(temp_type)_warning, \$(temp_type)_alarm.

b)、判断标准:

(1) 对上述所有操作, 执行成功.

(2) 任意\$(temp_type)_value 小于\$(temp_type)_warning 和\$(temp_type)_alarm.

4.7.2 DC/DC 电源 sensors:

a)、测试方法:依次执行步骤(1)~(3)

(1)获取系统DC/DC电流, 电压, 功率sensor列表, 记为list_dcdc_sensor_values.

(2)遍历\$(list_dcdc_sensor_values)中的 sensor, 对比\$(dcdc_type)_sensor_value 和\$(dcdc_type)_low_warning.

(3) 遍历\$(list_dcdc_sensor_values)中的 sensor, 对比\$(dcdc_type)_sensor_value 和\$(dcdc_type)_high_warning.

b)、判断标准:

(1) 对上述所有操作, 执行成功.

(2) 任意\$(dcdc_type)_sensor_value 大于 \$(dcdc_type)_low_warning.

(3) 任意\$(dcdc_type)_sensor_value 小于 \$(dcdc_type)_high_warning.

4.8 CPU 系统测试

4.8.1 CPU 信息显示

a)、测试方法:依次执行步骤(1)~(2)

(1) 读取 cpu 信息, cat /proc/cpuinfo.

(2) 抓取型号, 频率, cpu core 数量, 和期望值对比.

b)、判断标准:

(1) 型号和期望值相同.

(2) cpu core 数量和期望值相同.

(3) 频率和期望值相差小于 1%.

4.8.2 磁盘信息显示和状态检查

a)、测试方法:依次执行步骤(1)~(2)

(1) 使用 df -h 命令读取磁盘信息.

(2) 抓取磁盘大小, 磁盘使用率.

b)、判断标准:

(1) 磁盘大小和期望值相同.

(2) 磁盘使用率小于\$(disk_usage_warning_percentage).

4.8.3 内存信息显示和状态检查

a)、测试方法:依次执行步骤(1)~(2)

(1) 使用 free -t 命令读取内存信息.

(2) 抓取内存大小, free, cached, buffer 和 total.

b)、判断标准:

(1) 计算可用内存占比 \$(free_mem_percentage).

(2) \$(free_mem_percentage)应该大于\$(free_mem_percentage_expect).

4.8.4 PCIe 总线检查

a)、测试方法:依次执行步骤(1)~(4)

(1) 使用 lspci 命令扫描系统中所有 PCI 总线上的设备.

(2) 输出记为\$(pcie_devices).

(3) 对比\$(pcie_devices)和\$(pcie_devices_expect)

(4) 对于关键设备[FPGA, 交换芯片, CPU 网卡], 依次进行带宽检查: 对比 FPGA 的 speed 和\$(fpga_speed), width 和\$(fpga_width); 对比交换芯片的 speed 和\$(switch_speed), width 和\$(switch_width); 对比网卡的 speed 和\$(nic_speed), width 和\$(nic_width).

b)、判断标准:

(1) 步骤(3)对比, 一致

(2) 步骤(4)对比, 一致.

4.8.5 LPC 读写 CPLD

a)、测试方法:依次执行步骤(1)~(4)

(1) 通过平台/产品工具, 写入 CPLD 的特定地址, 值为\$(test_value).

(2) 读取 CPLD 该地址的值, 记为\$(test_value_readback).

(3) 对比\$(test_value)和\$(test_value_readback)

(4) 对 LPC 可以访问的所有 CPLD, 重复步骤 (1)~步骤 (3).

b)、判断标准:

(1) 所有命令执行成功

(2) 步骤 3)对比结果一致.

4.8.6 LPC 读写 BMC

a)、测试方法:依次执行步骤(1)~(3)

(1) 通过平台/产品工具, 写入 BMC 的特定寄存器, 值为\$(test_value).

(2) 读取 BMC 该寄存器的值, 记为\$(test_value_readback).

(3) 对比\$(test_value)和\$(test_value_readback).

b)、判断标准:

(1) 所有命令执行成功

(2) 步骤 3)对比结果一致.

4.8.7 前面板 USB 接口

a)、测试方法: 依次执行步骤(1)~(6)

(1) 插入 U 盘, 并用 fdisk -l 命令查找 U 盘

(2) 挂载 U 盘

(3) 复制测试文件到 U 盘, md5 为\$(test_file_md5)

(4) 计算 U 盘该文件副本的 md5, 记为\$(test_file_md5_usb)

(5) 对比\$(test_file_md5)和\$(test_file_md5_usb)

(6) 删除 U 盘测试文件, 拔掉 u 盘

b)、判断标准:

(1) 所有命令执行成功

(2) 步骤 5)对比结果相同

4.8.8 光模块链路扫描

a)、测试方法: 遍历所有前面板端口

(1) 获取 sysfs 路径

(2) 检查/sys/bus/i2c/devices/目录下对应的 sysfs 路径是否存在

(3) 检查模块 presece 节点是否可读

(4) 检查对应的 eeprom 的 sysfs 路径是否存在

(5) 检查对应的 eeprom 是否可以读

(6) 检查 dev_class 可读可写

b)、判断标准

(1) /sys/bus/i2c/devices/i2c-\$(bus)目录存在

(2) 检查 presence 为 1 的端口, 对应的 eeprom 可读

(3) dev_class 读写成功.

4.8.9 I2C 链路扫描(PORT-CPLD)

a)、测试方法:

(1) 获取产品 I2C_CPLD 列表

(2) 遍历 PORT-CPLD 对应的总线和地址, 用 i2cget/i2cset 写入测试值 \$(test_value), 再读取该值

(3) 对比\$(test_value)和\$(test_value_readback)

b)、判断标准

(1) 所有命令执行成功

(2) 步骤 3)对比值相同

4.8.10 CPU GPIO(JTAG 链路控制)

a)、测试步骤

- (1) 升级 port-cpld 到测试版, \$(port_cpld_test)
- (2) 刷新
- (3) 完成后读取 port-cpld 版本, 记为\$(port_cpld_test_readback)
- (4) 对比\$(port_cpld_test)\$ (port_cpld_test_readback)
- (5) 升级 port-cpld 到原始版本

b)、判断标准

- (1) 上述命令执行成功
- (2) 步骤 4)对比, 结果一致

4.8.11 SVID 测试(cpu 调压)

a)、测试方法

- (1) 通过平台/产品工具设置交换芯片电压\$(switch_vol_test)
- (2) 读取交换芯片电压\$(switch_vol_test_readback)
- (3) 对比\$(switch_vol_test)和\$(switch_vol_test_readback)

b)、判断标准

- (1) 命令执行成功
- (2) 步骤 3)对比结果一致

4.8.12 RTC 测试

a)、测试方法

- (1) 获取系统时间, 记为\$(time_start)
- (2) 程序延时固定时间\$(delay_interval)
- (3) 获取系统时间, 记为\$(time_end)
- (4) 计算\$(time_end)-\$(time_start), 记为\$(delta_interval)
- (5) 对比\$(delay_interval)和\$(delta_interval)

b)、判断标准

- (1) 命令执行成功
- (2) 步骤(5)差值不超过 1 秒

4.8.13 时区

a)、测试方法

- (1) 用 date 命令获取时区, 记为\$(time_zone)
- (2) 对比\$(time_zone)和\$(time_zone_expected)

b)、判断标准

- (1) 命令执行成功
- (2) 步骤(2)对比结果一致

4.8.14 内存测试

a)、测试方法: 使用 memtester 工具进行测试

b)、判断结果: memtester 的结果为 pass

4.9 可替换部件

4.9.1 风扇信息

a)、测试方法

(1) 获取风扇数量, 记为\$(fan_count)

(2) 遍历所有风扇, 执行以下

I) 获取风扇 FRU 信息

II) 检查 key

III) 检查状态信息

b)、判断标准

1) 所有命令执行成功

2) 步骤 2-a), key 必须包含[PartNumber, SerialNumber, FanName, Airflow, Presence]

3) PartNumber, SerialNumber, FanName 非空且不为相同字符

4) Presence 为 True, Airflow 为与系统风向相同

5) \$(fan_count)与\$(system_fan_count)相同

4.9.2 风扇状态

a)、测试方法

(1) 获取风扇数量

(2) 遍历所有风扇, 执行以下步骤

I) 获取 rotor 个数

II) 获取每个 rotor 的 speed, running_state, alarm_status

III) 获取每个 rotor 的 speed 范围

IV) 对比 speed 和 speed 范围

V) 检查 running_state, alarm_status

b)、判断标准

(1) 风扇数量和 \$(system_fan_count)一致

(2) 上述命令执行成功

(3) 每个 rotor 的 running_state 为 True, alarm_status 为 False

(4) 每个 rotor 的 speed 都在 speed 范围之内)

4.9.3 风扇控制

a)、测试方法

1) 停止风扇调速控制守护进程

2) 设置风扇转速到\$(fan_speed_test1)

3) 等待固定时间(3 秒) 读取 风扇转速, 记为\$(fan_speed_readback)

4) 对比风扇转速\$(fan_speed_readback)和\$(fan_speed_test1)

5) 替换 \$(fan_speed_test1)为 \$(fan_speed_test2), \$(fan_speed_test3),

重复执行步骤 2)~步骤 4)

6) 重新启动风扇控制守护进程

b)、判断标准

1) 所有命令执行成功

2) 步骤 4)对比, 一致

4.9.4 电源信息

a)、测试方法

1) 获取 PSU 数量, 记为\$(psu_count)

2) 遍历所有 PSU, 执行以下

a) 获取 PSU FRU 信息

b) 检查 key

c) 检查状态信息

b)、判断标准

1) 所有命令执行成功

2) 步骤 2-a), key 必须包含[PartNumber, SerialNumber, PsuName, Airflow, Presence]

3) PartNumber, SerialNumber, FanName 非空且不为相同字符

4) Presence 为 True, Airflow 为与系统风向相同

5) \$(psu_count)与\$(system_psu_count)相同

4.9.5 电源状态

a)、测试方法

1) 获取 PSU 数量

2) 遍历所有 PSU, 执行以下

I) 读取 PSU input 状态, 包括 in_status, in_power, in_voltage, in_current

II) 读取 PSU input sensor 的范围

III) 读取 PSU output 状态, 包括 out_status, out_power, out_voltage, out_current

IV) 读取 PSU output sensor 的范围

b)、判断标准

1) \$(psu_count) 和 \$(system_psu_count)一致

2) in_status 应为 True, in_power, in_voltage, in_current 分别处在 in_power_range, in_voltage_range, in_current_range 之内

3) out_status 应为 True, out_power, out_voltage, out_current 分别处在 out_power_range, out_voltage_range, out_current_range 之内

4.10 管理网口

4.10.1 测试方法

- 1) 配置管理网口 ip 地址
- 2) ifconfig eth0 查看 link 状态, 查看速度, 记为\$(link_speed)
- 3) 判断\$(link_speed)是否 1Gbps
- 4) ping 同一个局域网的服务器

4.10.2 判断标准

- 1) 步骤 3), 结果应该是 1Gbps
- 2) ping 结果应该没有丢包

4.11 光模块和 DAC

4.11.1 光模块信号位和 eeprom 读取

a)、测试方法

- (1) 使用 sfputil show presence 命令读取光模块在位信息
- (2) 使用 sfputil show eeprom -d 命令读取光模块信息
- (3) 遍历所有端口, 执行以下: 对比 presence 为 Yes 的模块是否和能正确读取 eeprom 信息

b)、判断标准

- (1) 所有命令正确执行
- (2) 步骤(3),presence 为 yes 的模块, eeprom 内容必须可以正确读取并解析
- (3) eeprom 需要包含正确的媒体类型, ID, 长度, vendor 信息等字段

4.12 数据平面

4.12.1 Snake 打流测试

a)、测试方法

- 1) 端口外部用线缆连接成 snake
- 2) 通过 CPU 发包测试
- 3) 读取全部端口统计
- 4) 更改 packet size, 重复步骤 2)步骤 3)

b)、判断标准

- 1) 所有命令执行成功
- 2) 检查步骤 3)中的端口统计, 没有丢包统计, tx_pkt, rx_pkt 都有统计

4.13 逻辑器件测试

4.13.1 CPU 读写 CPLD 寄存器

a)、测试步骤

- 1) 通过平台/产品接口向\$(cpld_type)的特定寄存器写入值\$(test_value)
- 2) 读取该 cpld 的同一个寄存器的值, 记为\$(test_value_readback)
- 3) 对比\$(test_value)和\$(test_value_readback)
- 4) 对系统支持从 CPU 读写的 CPLD, 都执行上述步骤 1)~步骤 3)

b)、判断标准

- 1) 所有命令正确执行
- 2) 步骤 3)的对比结果一致

4.13.2 CPU 通过 PCIe 读写 FPGA 寄存器

a)、测试步骤

- 1) 通过 sysfs 对 FPGA 读取值 , hexdump
/sys/bus/pci/devices/\${device_node}/config
- 2) 通过 sysfs 对 FPGA 读取值 ,
/sys/bus/pci/devices/\${device_node}/FPGA/debug, 值为\$(test_value)
- 3) 读取步骤 2)写入的值, 记为\$(test_value_readback)
- 4) 对比\$(test_value)和\$(test_value_readback)

b)、判断标准

- 1) 所有命令正确执行
- 2) 步骤 4)的对比结果一致
- 3) 步骤 1)能正确读取 config 的内容

www.ODCC.org.cn

本规范用词说明

为便于在执行本规范条文时区别对待，对要求严格程度不同的用词说明如下：

1) 表示很严格，非这样做不可的用词：

正面词采用“必须”，反面词采用“严禁”。

2) 表示严格，在正常情况下均应这样做的用词：

正面词采用“应”，反面词采用“不应”或“不得”。

3) 表示允许稍有选择，在条件许可时首先应这样做的用词：

正面词采用“宜”，反面词采用“不宜”；

表示有选择，在一定条件下可以这样做的用词，采用“可”。

本规范中指明应按其他有关标准、规范执行的写法为“应符合...的规定”或“应按...执行”

www.ODCC.org.cn

本页为规范最后一页



WWW.ODCC.ORG.CN

